

Predicting Heart Disease Risk Using Machine Learning Models

Name: vedha.s

Branch: B.Sc Computer science

Introduction :

Heart disease is one of the leading causes of death worldwide. Early detection of individuals at risk can help in timely intervention and treatment. This project aims to develop a machine learning model to predict whether a patient is at risk of heart disease based on various health indicators. Using the UCI Heart Disease Dataset, we analyze the data, visualize important trends, and compare different machine learning models—Logistic Regression, Decision Tree, and Random Forest—to determine the most accurate classifier for heart disease prediction.

Requirements ::

Libraries:

Python 3.x

NumPy – for numerical operations

Pandas – for handling datasets

Matplotlib & Seaborn – for data visualization

Scikit-learn – for machine learning models and evaluation

Dataset:

UCI Heart Disease Dataset (CSV format)

Implementation:

1. Data Analysis: a. Import the dataset b. Get information about the dataset c. Find the

correlation between all fields .

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, precision_recall_fscore_support
```

Load dataset:

```
df = pd.read_csv("heart1.csv") # Update path if needed
```

Display basic info:

```
print(df.info())
print(df.describe())
```

Correlation matrix:

```
correlation_matrix = df.corr()
print(correlation_matrix)
```

2. Data Visualization: a. Visualize the number of patients having a heart disease and not

having a heart disease b. Visualize the age and whether a patient has disease or not c. Visualize correlation between all features using a heatmap.

Countplot of target variable:

```
plt.figure(figsize=(6, 4))
sns.countplot(x=df["target"], palette="viridis")
plt.title("Number of Patients with and without Heart Disease")
plt.xlabel("Heart Disease (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()
```

Age distribution by heart disease:

```
plt.figure(figsize=(8, 5))
sns.boxplot(x=df["target"], y=df["age"], palette="coolwarm")
plt.title("Age Distribution by Heart Disease")
plt.xlabel("Heart Disease (0 = No, 1 = Yes)")
plt.ylabel("Age")
plt.show()
```

Heatmap of correlation matrix:

```
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```

3. Logistic Regression: a. Build a simple logistic regression model: i. Divide the dataset in

70:30 ratio ii. Build the model on train set and predict the values on test set iii. Build the confusion matrix and get the accuracy score

Define features and target variable:

```
X = df.drop(columns=["target"])
```

```
y = df["target"]
```

Split data into train (70%) and test (30%) sets:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
```

Standardize the features:

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

LOGISTIC REGRESSION :

```
log_reg = LogisticRegression()
```

```
log_reg.fit(X_train_scaled, y_train)
```

```
y_pred_log_reg = log_reg.predict(X_test_scaled)
```

```
log_reg_acc = accuracy_score(y_test, y_pred_log_reg)
```

```
log_reg_cm = confusion_matrix(y_test, y_pred_log_reg)
```

```
log_reg_report = classification_report(y_test, y_pred_log_reg)
```

```
print("\nLogistic Regression Accuracy:", log_reg_acc)
```

```
print("Confusion Matrix:\n", log_reg_cm)

print("Classification Report:\n", log_reg_report
```

4. Decision Tree: a. Build a decision tree model: i. Divide the dataset in 70:30 ratio ii. Build the model on train set and predict the values on test set iii. Build the confusion matrix and calculate the accuracy iv. Visualize the decision tree using the Graphviz package.

DECISION TREE :

```
dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

dt_acc = accuracy_score(y_test, y_pred_dt)

dt_cm = confusion_matrix(y_test, y_pred_dt)

dt_report = classification_report(y_test, y_pred_dt)

print("\nDecision Tree Accuracy:", dt_acc)

print("Confusion Matrix:\n", dt_cm)

print("Classification Report:\n", dt_report)
```

Visualize Decision Tree:

```
plt.figure(figsize=(12, 6))

plot_tree(dt, filled=True, feature_names=X.columns, class_names=["No Heart Disease",
"Heart Disease"])

plt.title("Decision Tree Visualization")

plt.show()
```

5. Random Forest: a. Build a Random Forest model: i. Divide the dataset in 70:30 ratio ii. Build the model on train set and predict the values on test set iii. Build the confusion matrix and calculate the accuracy iv. Visualize the model using the Graphviz package .

RANDOM FOREST:

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf.fit(X_train, y_train)
```

```
y_pred_rf = rf.predict(X_test)
```

```
rf_acc = accuracy_score(y_test, y_pred_rf)
```

```
rf_cm = confusion_matrix(y_test, y_pred_rf)
```

```
rf_report = classification_report(y_test, y_pred_rf)
```

```
print("\nRandom Forest Accuracy:", rf_acc)
```

```
print("Confusion Matrix:\n", rf_cm)
```

```
print("Classification Report:\n", rf_report)
```

6. Select the best model a. Print the confusion matrix of all classifiers b. Print the classification report of all classifiers c. Calculate Recall Precision and F1 score of all the models d. Visualize confusion matrix using heatmaps e. Select the best model based on the best accuracies.

Precision, Recall, F1-score:

```
log_reg_scores = precision_recall_fscore_support(y_test, y_pred_log_reg, average='binary')
```

```
dt_scores = precision_recall_fscore_support(y_test, y_pred_dt, average='binary')
```

```
rf_scores = precision_recall_fscore_support(y_test, y_pred_rf, average='binary')
```

Print scores:

```
print("\nModel Comparison:")

print("Logistic Regression - Precision:", log_reg_scores[0], "Recall:", log_reg_scores[1], "F1-score:", log_reg_scores[2])
print("Decision Tree - Precision:", dt_scores[0], "Recall:", dt_scores[1], "F1-score:", dt_scores[2])
print("Random Forest - Precision:", rf_scores[0], "Recall:", rf_scores[1], "F1-score:", rf_scores[2])
```

Confusion Matrix Heatmaps:

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, model, cm, title in zip(axes, ["Logistic Regression", "Decision Tree", "Random Forest"],
[log_reg_cm, dt_cm, rf_cm], ["LR", "DT", "RF"]):

    sns.heatmap(cm, annot=True, fmt="d", cmap="coolwarm", ax=ax)

    ax.set_title(f"Confusion Matrix - {model}")

    ax.set_xlabel("Predicted")

    ax.set_ylabel("Actual")

plt.show()
```

Final Model Selection:

```
best_model = max([(log_reg_acc, "Logistic Regression"), (dt_acc, "Decision Tree"), (rf_acc, "Random Forest")])
print(f"\nBest Model: {best_model[1]} with Accuracy {best_model[0]}")
```

Conclusion : This project successfully implemented three machine learning model for heart disease

prediction: Logistic Regression, Decision Tree, and Random Forest. Based on evaluation metrics like accuracy, precision, recall, and F1-score, the best-performing model was selected. The Random Forest model performed the best after hyperparameter tuning.